# Assignment 3: LLM Recommender System Report

## 1. Project Overview

This project implements a local Retrieval-Augmented Generation (RAG) recommender for Steam games. Given a natural-language query (for example: "cozy farming game with exploration"), the system retrieves relevant games and returns:

- a ranked list of game matches
- a short natural-language explanation of why they fit

The pipeline combines semantic retrieval, metadata/tag signals, and LLM-based reranking.

## 2. What Was Implemented

The recommender in recommender.py includes:

- Query analysis and expansion:
    - token extraction
    - Local alias expansion (tags and intent): the system first expands user query terms with predefined local alias mappings to improve recall. In the initial build, when writing relaxed in the prompt it would not flag cozy as a positive tag.
    - LLM-assisted expansion (tags and intent clusters): the LLM analyses the query and proposes additional positive tags, reverse tags, and intent clusters. During scoring, positive tag matches receive boosts, reverse-tag matches receive penalties, and multiple positive tag hits get cumulative bonus effects (up to a capped maximum).
- Candidate retrieval:
    - semantic retrieval via FAISS
    - keyword fallback retrieval if index is unavailable. This was the initial retrieval functionality and has been kept as a fallback method when the FAISS index is unavailable.
- Hybrid pre-ranking:
    - semantic similarity weight: 0.35
    - tag match weight: 0.30 (includes ordered-tag component inside tag scoring) Ordered means that tags that are mentioned first in the prompt have more weight (about 20% of the tag score). e.g. If the prompt is "I want a violent survival game, set in a post-apocalyptic world." Violent and survival are the most important tags, so the results could include games not set in the preferred setting but respecting the genre. On the other hand, if the prompt is "I want a post-apocalyptic game with survival and violence." The setting would be most important.

    - popularity prior weight: 0.20 (calculated from review volume, recommendation count, concurrent players, and review sentiment.)
    - review-volume weight: 0.15 (A bit like the popularity but more focused on community feedback.)
    - positive-tag bonus: +0.07 per matching tag (capped at +0.45)
    - reverse-tag penalty: -0.04 per matching tag (capped at -0.25)
    - perspective/intent boost: up to +0.12 (perspective match, e.g. First person. Genre match and interaction match e.g. Co-op or PvP.)
- Dynamic anchor recall:
    - The LLM suggests popular genre-relevant anchor games, then the system verifies them against the database and adds valid matches to the candidate pool. This is done because the initial system was only recommending niche games. e.g. If the prompt is "I want a story driven turn-based RPG" the natural recommendation would be Baldur's Gate 3, however the system was suggesting games that had a better semantic fit, e.g. RPG in the game title.
- LLM reranking:
    - candidate reranking using gameplay loop fit, mechanic friction fit, and player-fantasy fit
    - blended with base retrieval signal
    - final weighted score components:
    - semantic match: 0.07
    - gameplay loop fit: 0.30
    - mechanic friction fit: 0.10
    - player fantasy fit: 0.16
    - tag metadata fit: 0.32
    - community sentiment: 0.02
    - popularity success prior: 0.03
- Answer generation:
    - concise explanation generated from top ranked games
    - LLM prompt used for answer generation:
    - System prompt: "You write concise recommendation explanations grounded in supplied game metadata and reviews. Only mention games from top_matches, and mention them in the same order provided. Do not add or reference any other game titles. Do not invent mechanics or features not present in context. Keep it under 90 words."
    - fallback explanation when LLM output is unavailable

# 3. Data and Indexing Approach

Data source:

- SQLite database: steam_games_reviews_25.sqlite
- tables include game metadata and reviews

Offline indexing (build_index.py):

1. Load game metadata from SQLite
2. Load recent English review snippets per game
3. Build one combined text document per game with:
    - title
    - genres
    - tags
    - description
    - truncated recent review summary
1. Encode documents with a sentence-transformer
2. Build and save FAISS index plus metadata cache

Saved artifacts:

- embeddings/faiss.index
- embeddings/metadata.pkl

At runtime, the recommender loads these cached artifacts instead of recomputing embeddings.

# 4. Technology Stack (Implemented)

Backend and serving:

- Python
- Flask

Retrieval and ranking:

- sentence-transformers: chosen for strong out-of-the-box semantic embeddings and simple integration with local models such as all-MiniLM-L6-v2.
- FAISS (faiss-cpu): chosen for fast nearest-neighbour vector search with low overhead, making query-time retrieval efficient on a local machine.
- NumPy: chosen for efficient numeric operations (normalization, vector handling, score shaping) required for embedding and ranking pipelines.

LLM interaction:

- Ollama local API via HTTP requests: chosen to run LLM inference fully locally (no paid cloud dependency), with controllable latency and model switching.
- requests: chosen as a lightweight and reliable HTTP client for direct API calls to Ollama endpoints.

Data storage:

- SQLite

Environment/package management:

- uv

# 5. Models Used / Tried (Short Summary)

Embedding model implemented:

- all-MiniLM-L6-v2 (default embedding model)

LLM model implemented as default for reranking and answer generation:

- gemma3:4b via Ollama

Alternative/fallback models noted in project instructions:

- phi4-mini
- qwen2.5

Cloud based LLM option:

OpenAI-compatible NVIDIA endpoint

Pros:

- stronger reasoning and language quality on many prompts
- no local model hosting burden
- easy integration through OpenAI-compatible clients

Cons:

- external API dependency (internet + service availability)
- potential usage cost and token budgeting concerns 40RPM on the free plan.

Why not used here:

- Both were created but the Ollama implementation gave better results during testing.

# 6. Strengths and Limitations

Strengths:

- Efficient offline indexing + fast online retrieval
- Hybrid ranking (semantic + metadata + popularity + sentiment)
- Robust fallback behavior when LLM/index is unavailable
- Fully local execution path

Limitations:

- Quality depends on local LLM performance and prompt reliability
- Ranking weights are heuristic and require tuning/evaluation. It was done arbitrarily during testing

# 7. Conclusion and Lessons Learned

The implemented system is a complete LLM-driven recommender. In practice, the strongest results came from combining semantic retrieval with structured ranking signals (tags, popularity, sentiment, and intent boosts) rather than relying on embeddings or LLM output alone.

Lessons learned:

- A hybrid approach is more robust than a single-method approach; semantic similarity alone can over-prioritize niche lexical matches.
- Query understanding matters significantly; local aliases plus optional LLM-assisted tag/intent expansion improved recall and relevance.
- Controlled weighting is essential; small adjustments to tag, popularity, and boost/penalty terms noticeably changed ranking behaviour.
- Anchor recall helped reduce false-niche recommendations by reinserting high-confidence, genre-appropriate popular titles.

In conclusion, recommender systems constantly must juggle between giving hyper accurate or popular but broadly matching recommendations. To tackle this in our system we used the anchoring recommendations via LLM to make sure that the final recommendations are a mix of hyper accurate ones and popular ones.